POSTGRESQL ADMINISTRATOR - TÀI LIỆU ÔN TẬP BẢN CHẤT

Dành cho Data Engineer / Junior DBA - thực hành trên Mascot self-host

Mục tiêu: Sau khi đọc lại tài liệu này, bạn phải hiểu vì sao PostgreSQL hoạt động như vậy, biết quan sát hệ thống, quản lý quyền, kiểm tra connection, backup/restore, hiểu WAL/VACUUM và đọc execution plan cơ bản.

Cách dùng: Đọc theo thứ tự. Với mỗi phần, hãy tự trả lời 3 câu: (1) thành phần này giải quyết vấn đề gì, (2) lệnh nào dùng để quan sát/thao tác, (3) output nào chứng minh mọi thứ đang đúng.

Phần	Trọng tâm
I	PostgreSQL Administration là gì
II	Kiến trúc server - cluster - database - schema - table
III	Service, data directory và initdb
IV	postgresql.conf
V	pg_hba.conf và authentication
VI	Role, user, owner, privilege
VII	Connection/session và pg_stat_activity
VIII	Backup/Restore
IX	WAL và durability
X	MVCC, dead tuples, VACUUM/autovacuum
XI	EXPLAIN/EXPLAIN ANALYZE
XII	Troubleshooting theo layer
XIII	Lab checklist và câu hỏi mentor/phỏng vấn

I. PostgreSQL Administration khác SQL như thế nào?

SQL tập trung vào việc truy vấn và thay đổi dữ liệu. PostgreSQL Administration tập trung vào việc giữ cho database server chạy đúng, an toàn, có thể phục hồi và đủ hiệu năng.

SQL / Developer	PostgreSQL Administration / DBA
SELECT, INSERT, UPDATE, DELETE	Service có chạy không?
JOIN, GROUP BY, window function	Ai được connect? Từ đâu?
Viết query đúng	Port/listen/authentication đúng chưa?
Thiết kế câu lệnh SQL	Connection có quá nhiều không?
Đọc dữ liệu	Backup/restore có hoạt động không?
Tối ưu logic query	VACUUM/WAL/query plan có vấn đề không?

> **Liên hệ Mascot:** FastAPI chỉ chạy được ổn định khi PostgreSQL 18 trên Oracle Linux đang active, backend có đúng credentials/quyền, network cho phép container tới DB, schema đã migrate và DB không gặp vấn đề tài nguyên.

II. Kiến trúc PostgreSQL - hiểu đúng từ ngoài vào trong

Hãy hình dung PostgreSQL theo cây phân cấp sau:

```
PostgreSQL Server / Instance
├── Database cluster (cùng data directory)
│   ├── postgres
│   ├── mascot
│   ├── template0
│   └── template1
├── Roles
└── Connections / Sessions

Trong database mascot:
mascot
└── schema public
    ├── users
    ├── vocabulary_entries
    └── ...
```

1. PostgreSQL server / instance

Đây là server process đang chạy và nhận connection. Trên Oracle Linux, service được quản lý bằng systemd.

```
sudo systemctl status postgresql-18
```

> **Điểm cần hiểu:** Database không phải một file độc lập đang tự chạy. PostgreSQL server process mới là thứ lắng nghe connection và quản lý nhiều database bên trong cluster.

2. Database cluster

Trong PostgreSQL, "database cluster" thường nghĩa là tập hợp database dùng chung một PostgreSQL instance và một data directory. Nó không đồng nghĩa với cluster nhiều máy chủ.

```
sudo /usr/pgsql-18/bin/postgresql-18-setup initdb
```

Lệnh initdb khởi tạo cluster: tạo data directory, system catalogs, database template và file cấu hình. Nó chưa phải là CREATE DATABASE mascot.

Thao tác	Ý nghĩa
initdb	Khởi tạo toàn bộ PostgreSQL cluster
CREATE DATABASE mascot	Tạo một database bên trong cluster
CREATE SCHEMA	Tạo namespace bên trong database

CREATE TABLE	Tạo object dữ liệu bên trong schema

3. Database, schema, table

Một connection PostgreSQL luôn kết nối tới một database cụ thể. Trong database có schema; trong schema có table, view, sequence, function... Schema giúp tổ chức namespace và phân quyền.

```
\l        -- xem database
\c mascot -- chuyển database
\dn       -- xem schema
\dt       -- xem table
```

III. Service, data directory và các file quan trọng

1. Service lifecycle

```
sudo systemctl status postgresql-18
sudo systemctl start postgresql-18
sudo systemctl stop postgresql-18
sudo systemctl restart postgresql-18
sudo systemctl enable postgresql-18
systemctl is-enabled postgresql-18
```

start/stop/restart tác động trạng thái hiện tại. enable quyết định service có tự khởi động khi boot hay không.

> **Production habit:** Trước khi restart PostgreSQL, nên biết mình đang sửa gì và xem log nếu đang xử lý lỗi. Restart vô thức có thể che mất symptom ban đầu và làm gián đoạn service.

2. Data directory

```
SHOW data_directory;
```

Data directory chứa dữ liệu vật lý, catalog, WAL, config và metadata của cluster. Không chỉnh sửa/xóa file trực tiếp trong đó nếu không hiểu rõ.

```
SHOW config_file;
SHOW hba_file;
```

IV. postgresql.conf - PostgreSQL server vận hành như thế nào?

postgresql.conf chứa phần lớn tham số cấu hình của server. Ba tham số quan trọng trong lab là listen_addresses, port và max_connections.

1. listen_addresses

```
SHOW listen_addresses;
```

Nó quyết định PostgreSQL lắng nghe trên network interface nào. Nếu chỉ listen 127.0.0.1 thì chỉ client trên chính Linux host có thể mở TCP connection tới đó.

```
listen_addresses = '*'
```

> **Không được nhầm:** listen_addresses='*' chỉ nói rằng PostgreSQL chịu nghe trên các interface. Nó KHÔNG tự động cấp quyền đăng nhập cho mọi client. Quyền kết nối còn qua pg_hba.conf, credentials và privilege.

2. port

```
SHOW port;
```

Port mặc định PostgreSQL là 5432. IP xác định máy; port xác định service trên máy.

```
192.168.1.166:22   -> SSH
192.168.1.166:5432 -> PostgreSQL
192.168.1.166:8000 -> Backend
192.168.1.166:5173 -> Frontend
```

3. max_connections

```
SHOW max_connections;
```

Đây là giới hạn connection đồng thời. Application thường dùng connection pool, vì vậy một user web không tương ứng chính xác một DB connection. Khi pool/app mở quá nhiều connection, server có thể báo "too many connections".

> **Đi làm:** Không nên chỉ tăng max_connections để chữa cháy. Cần xem connection pool, idle connection, workload và tài nguyên RAM trước.

V. pg_hba.conf - ai được kết nối, từ đâu và bằng cách nào?

HBA = Host-Based Authentication. File này kiểm soát rule authentication theo loại connection, database, role, source address và auth method.

```
host    mascot    mascot_user    172.16.0.0/12    scram-sha-256
```

Cột	Ý nghĩa
host	TCP/IP connection
mascot	Database được phép connect
mascot_user	Role/user
172.16.0.0/12	Dải source IP cho phép
scram-sha-256	Phương pháp xác thực password

> **Đọc rule thành câu:** Cho role mascot_user từ mạng Docker 172.16.0.0/12 kết nối database mascot qua TCP/IP và authenticate bằng SCRAM-SHA-256.

1. SCRAM-SHA-256

SCRAM là cơ chế challenge-response cho password authentication. Ở mức quản trị cơ bản, cần nhớ đây là phương pháp password auth hiện đại và an toàn hơn cơ chế MD5 cũ.

2. Tư duy hai lớp config

```
Client
  ↓
postgresql.conf: server có LISTEN ở interface/port này không?
  ↓
pg_hba.conf: source/user/database/auth method này có được phép không?
  ↓
Password / role / privilege
  ↓
Database session
```

VI. Role, User, Owner và Privilege

PostgreSQL dùng khái niệm ROLE làm nền tảng. USER thực chất là role có LOGIN.

```
CREATE ROLE report_role;
CREATE USER app_user WITH PASSWORD '...';
-- gần tương đương: CREATE ROLE app_user LOGIN PASSWORD '...';
```

1. Superuser

Role postgres thường là superuser. Nó có quyền rất rộng, vì vậy application backend không nên dùng postgres để kết nối thường xuyên.

> **Nguyên tắc:** Least privilege - application chỉ nên có những quyền cần cho nghiệp vụ. Nếu backend bị compromise nhưng DB role bị giới hạn, phạm vi thiệt hại được giảm đáng kể.

2. Owner

```
CREATE DATABASE mascot OWNER mascot_user;
```

Owner có quyền quản lý object mà role đó sở hữu. Ownership khác với privilege được GRANT từ người khác.

3. Privilege

```
GRANT CONNECT ON DATABASE mascot TO report_user;
GRANT USAGE ON SCHEMA public TO report_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO report_user;
REVOKE INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public FROM report_user;
```

Privilege	Ý nghĩa thường gặp
CONNECT	Được mở connection vào database
USAGE on schema	Được truy cập object name trong schema
SELECT	Đọc row
INSERT	Thêm row
UPDATE	Sửa row
DELETE	Xóa row
CREATE	Tạo object ở phạm vi được cấp

4. Lệnh quan sát

```
\du
\l
\dn+
\dp
```

VII. Connection, Session và pg_stat_activity

Connection là kênh client-server. Session là trạng thái PostgreSQL duy trì cho connection đó: user, database, transaction state, query hiện tại, session settings...

```
SELECT pid, usename, datname, client_addr, state, query
FROM pg_stat_activity;
```

Cột	Ý nghĩa
pid	Process ID phía PostgreSQL backend process
usename	Role đang dùng
datname	Database đang kết nối
client_addr	IP client
state	active, idle, idle in transaction...
query	Query gần nhất/đang chạy

1. active và idle

active nghĩa là session đang thực thi query. idle nghĩa là connection vẫn mở nhưng hiện không chạy query. idle không tự động là lỗi; connection pool thường duy trì idle connections để tái sử dụng.

> **Cảnh báo:** "idle in transaction" đáng chú ý hơn idle thông thường vì transaction vẫn mở; nó có thể giữ lock hoặc giữ old row versions lâu.

2. Liên hệ Docker

Backend container có thể xuất hiện trong pg_stat_activity với client_addr thuộc Docker bridge subnet, ví dụ 172.18.x.x. Đây là bằng chứng network path từ container tới PostgreSQL host đang hoạt động.

```
SELECT pid, usename, datname, client_addr, state
FROM pg_stat_activity
WHERE datname = 'mascot';
```

VIII. Backup và Restore - kỹ năng bắt buộc của quản trị DB

Backup chỉ có ý nghĩa khi bạn đã chứng minh restore hoạt động. Một file dump chưa từng restore thử chưa nên được coi là chiến lược backup hoàn chỉnh.

1. Plain SQL dump

```
pg_dump -U mascot_user -h localhost -d mascot > mascot.sql
psql -U mascot_user -h localhost -d mascot_restore < mascot.sql
```

2. Custom format

```
pg_dump -U mascot_user -h localhost -d mascot -Fc -f mascot.dump
pg_restore -U mascot_user -h localhost -d mascot_restore mascot.dump
```

Format	Restore	Đặc điểm
Plain SQL	psql	Human-readable, dễ xem SQL
Custom (-Fc)	pg_restore	Linh hoạt hơn, hỗ trợ chọn object và parallel restore ở các trường hợp phù hợp

3. Lab an toàn

```
mkdir -p ~/pg_backups
pg_dump -U mascot_user -h localhost -d mascot -Fc -f ~/pg_backups/mascot.dump
ls -lh ~/pg_backups/mascot.dump
sudo -u postgres createdb -O mascot_user mascot_backup_test
pg_restore -U mascot_user -h localhost -d mascot_backup_test ~/pg_backups/mascot.dump
psql -U mascot_user -h localhost -d mascot_backup_test
```

> **Verify:** Trong database restore, dùng \dt, SELECT COUNT(*) và kiểm tra schema/version quan trọng. Mục tiêu là chứng minh backup có thể dùng để phục hồi, không chỉ chứng minh file tồn tại.

4. Backup logic không phải copy data directory

pg_dump tạo logical backup ở mức object/data. Copy thô data directory khi PostgreSQL đang chạy không tương đương một logical backup an toàn; physical backup cần quy trình riêng.

IX. WAL - nền tảng durability, crash recovery và replication

WAL = Write-Ahead Logging. Nguyên tắc cốt lõi: thông tin thay đổi cần được ghi vào WAL trước khi data page tương ứng bắt buộc phải được flush xuống storage.

```
Transaction thay đổi dữ liệu
        ↓
Ghi WAL record
        ↓
COMMIT có thể được xác nhận theo durability setting
        ↓
Data page được flush sau
        ↓
Crash? PostgreSQL replay WAL để recovery
```

WAL giúp PostgreSQL phục hồi trạng thái nhất quán sau crash và là nền tảng cho streaming replication/PITR.

> **Liên hệ ACID:** WAL là một phần quan trọng của cơ chế giúp đảm bảo Durability (D), nhưng durability còn liên quan fsync, storage và các setting khác.

```
SHOW data_directory;
sudo ls /var/lib/pgsql/18/data/pg_wal | head
```

> **Không làm:** Không xóa file trong pg_wal bằng tay để "giải phóng dung lượng". WAL tăng bất thường cần tìm nguyên nhân như replication slot, archive failure, long backup, workload... chứ không xóa thủ công.

X. MVCC, Dead Tuples, VACUUM và AUTOVACUUM

1. MVCC trực giác

PostgreSQL dùng Multi-Version Concurrency Control để nhiều transaction đọc/ghi đồng thời với ít blocking hơn. Khi UPDATE, PostgreSQL thường tạo row version mới thay vì overwrite đơn giản row cũ tại chỗ.

```
Version cũ: id=1, name='Dat'
UPDATE ...
Version mới: id=1, name='Dat New'

Sau khi không còn transaction nào cần version cũ -> version cũ trở thành dead tuple
```

2. VACUUM

VACUUM xử lý dead tuples để không gian có thể được tái sử dụng và duy trì metadata visibility cần cho MVCC. VACUUM thường không co file table về nhỏ ngay trên filesystem.

```
VACUUM table_name;
VACUUM (ANALYZE) table_name;
```

3. AUTOVACUUM

Autovacuum tự động chạy VACUUM/ANALYZE theo threshold. Tắt autovacuum trên production mà không có kế hoạch thay thế là rủi ro lớn.

```
SHOW autovacuum;
```

4. Quan sát dead tuples

```
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

> **Điểm bản chất:** UPDATE/DELETE nhiều -> tạo row versions cũ -> dead tuples tăng -> autovacuum phải theo kịp. Nếu không, table/index bloat và performance có thể xấu dần.

XI. EXPLAIN và EXPLAIN ANALYZE - nhìn cách PostgreSQL thực thi query

1. EXPLAIN

```
EXPLAIN
SELECT * FROM vocabulary_entries
WHERE cefr_level = 'B1';
```

EXPLAIN hiển thị execution plan mà planner chọn cùng estimated cost/rows/width. Nó chủ yếu là kế hoạch/ước lượng, không phải timing thực tế của query.

2. EXPLAIN ANALYZE

```
EXPLAIN ANALYZE
SELECT * FROM vocabulary_entries
LIMIT 20;
```

EXPLAIN ANALYZE thực sự chạy query và bổ sung actual time, actual rows, loops. Vì nó thực thi query thật, cần thận trọng với UPDATE/DELETE/INSERT.

Thuật ngữ	Hiểu đơn giản
Seq Scan	Đọc tuần tự table hoặc phần lớn table
Index Scan	Dùng index để tìm row
cost=a..b	Startup cost và total estimated cost; không phải milliseconds
actual time	Thời gian thực đo khi ANALYZE
rows	Số row ước lượng hoặc thực tế tùy phần plan

3. Seq Scan không phải lúc nào cũng xấu

Nếu table nhỏ hoặc query cần phần lớn rows, Seq Scan có thể rẻ hơn Index Scan. Tối ưu query không phải quy tắc "thấy Seq Scan là tạo index".

> **Đi làm:** Quan sát chênh lệch estimated rows và actual rows. Nếu lệch lớn, statistics có thể không phản ánh dữ liệu tốt hoặc query/data distribution có vấn đề.

XII. Troubleshooting PostgreSQL theo layer

Khi backend không connect DB, đừng sửa ngẫu nhiên. Đi từ thấp lên cao.

```
1. Process/service
2. Listen/port
3. Network path
4. pg_hba.conf
5. Credentials/authentication
6. Database/role/privilege
7. Schema/migration
8. Query/lock/performance
```

Symptom	Layer nghi ngờ	Kiểm tra
connection refused	Service/listen/port	systemctl status, ss -lntp
timeout	Network/firewall/routing	ping/curl/nc, firewall, IP
no pg_hba.conf entry	HBA rule	SHOW hba_file; xem source/user/db
password authentication failed	Credentials/auth	user/password, auth method
permission denied for table	Privilege/owner	\dp, GRANT, owner
relation does not exist	Schema/search_path/migration	\dt, alembic current
too many connections	Pool/max_connections	pg_stat_activity, pool config
query chậm	Plan/lock/I/O/stats	EXPLAIN ANALYZE, pg_stat_activity, locks

Ví dụ suy luận quan trọng

Nếu PostgreSQL trả về "password authentication failed", request đã đi qua network và tới được PostgreSQL để server có thể thực hiện authentication. Vì vậy lỗi chính lúc đó không còn là "port hoàn toàn không tới được" mà tập trung vào credentials/auth config.

XIII. Command cheat sheet để ôn nhanh

Mục tiêu	Lệnh
Vào psql với postgres	sudo -u postgres psql
Xem DB	\l
Xem roles	\du
Chuyển DB	\c mascot
Xem schema	\dn
Xem tables	\dt
Mô tả table	\d table_name
Thoát	\q
Reset query buffer	\r
Config file	SHOW config_file;
HBA file	SHOW hba_file;
Listen	SHOW listen_addresses;
Port	SHOW port;
Connections	SELECT ... FROM pg_stat_activity;
Dead tuples	SELECT ... FROM pg_stat_user_tables;
Backup custom	pg_dump ... -Fc -f file.dump
Restore custom	pg_restore ... -d db file.dump
Plan	EXPLAIN ...
Plan thực tế	EXPLAIN ANALYZE ...

XIV. Lab checklist - tự thực hành lại trong 60-90 phút

1. SSH vào Oracle Linux VM và xác nhận PostgreSQL service active.
2. Vào psql bằng postgres; xem \l, \du, chuyển vào mascot, xem \dn và \dt.
3. Chạy SHOW data_directory/config_file/hba_file/listen_addresses/port/max_connections.
4. Thoát psql và dùng ss -lntp để đối chiếu PostgreSQL đang listen port nào.
5. Xem rule pg_hba.conf liên quan mascot_user và tự đọc thành câu.
6. Xem pg_stat_activity khi backend Mascot đang chạy; xác định usename, client_addr, state.
7. Tạo test role, cấp SELECT hạn chế, kiểm tra quyền rồi REVOKE.
8. Backup mascot bằng pg_dump custom format.
9. Tạo mascot_backup_test và restore bằng pg_restore.
10. Kiểm tra table/count sau restore rồi xóa test database khi không cần.
11. Xem SHOW autovacuum và pg_stat_user_tables.
12. Chạy EXPLAIN và EXPLAIN ANALYZE trên một SELECT an toàn; đọc Seq Scan/Index Scan/cost/actual time.

XV. Câu hỏi mentor / phỏng vấn tự kiểm tra

- initdb tạo gì? Khác CREATE DATABASE như thế nào?
- Database cluster trong PostgreSQL nghĩa là gì?
- postgresql.conf và pg_hba.conf khác nhau ở nhiệm vụ nào?
- listen_addresses="*" có nghĩa ai cũng connect được không?
- Tại sao application không nên dùng role postgres?
- Role, user, owner và privilege khác nhau thế nào?

- active, idle và idle in transaction khác nhau ra sao?
- pg_stat_activity giúp admin làm gì?
- Backup có được coi là tốt nếu chưa restore test không?
- WAL giải quyết vấn đề gì? Liên quan gì tới crash recovery?
- Vì sao UPDATE/DELETE tạo dead tuples?
- VACUUM và autovacuum dùng để làm gì?
- EXPLAIN khác EXPLAIN ANALYZE thế nào?
- Seq Scan có phải lúc nào cũng xấu không?
- Nếu backend báo password authentication failed, bạn suy luận được gì về network path?
- Nếu server báo too many connections, vì sao không nên chỉ tăng max_connections?

XVI. Mental model cuối cùng - cách suy nghĩ như một junior DBA

```
Ứng dụng báo lỗi DB
        ↓
DB service có sống?
        ↓
Đang listen đúng IP/port?
        ↓
Network có đi tới?
        ↓
pg_hba cho phép?
        ↓
Credentials đúng?
        ↓
Role có quyền?
        ↓
Schema/migration đúng?
        ↓
Query có lock/chậm?
        ↓
Storage/WAL/VACUUM/resource có vấn đề?
```

Điều quan trọng nhất: Quản trị PostgreSQL không phải nhớ nhiều lệnh. Kỹ năng cốt lõi là biết hệ thống được chia thành layer nào, quan sát đúng layer, đọc output để thu hẹp nguyên nhân và chỉ thay đổi khi đã có giả thuyết rõ ràng.

HẾT - dùng tài liệu này để ôn lại trước khi thực hành trên VM hoặc trước buổi trao đổi với mentor.